

Hand-On-Learning: Split the Chain – Design and Deploy Your First LangChain Service

Learning Objectives

By the end of this activity, learners will be able to:

- Identify logical microservice boundaries in a monolithic LangChain Q&A pipeline.
- Design a gRPC interface (.proto) for one LangChain component (retriever, LLM, or summarizer).
- Implement a gRPC microservice in Python that wraps LangChain logic.
- Test the service locally using grpcurl or Postman and reflect on trade-offs of their boundary decisions.



Description

In this hands-on practice, you'll act as a backend developer at **TechDocs** tasked with proving that splitting a monolithic LangChain Q&A system into microservices is viable. You'll choose one component (retriever, LLM gateway, or summarizer), design a gRPC interface for it, implement it as a standalone Python + LangChain service, and test it locally. Your work becomes the **blueprint** for future microservices that will help TechDocs scale from 50 to 5,000+ users without constant freezes and full redeploys.

Structure of the Activity

Title

Split the Chain – Design and Deploy Your First LangChain Service

Dataset information / Resources Required

Starter code:

techdocs_monolith.py – monolithic LangChain Q&A script (retriever → LLM → summarizer).

Optional stubs / mocks:

→ A simple in-memory or mock vector store for the retriever.

→ A fake or test LLM endpoint (or OpenAI key, if available).

Tools used:

→ **Language / Runtime: Python 3.10+**

→ **Libraries:**

→ langchain (core + relevant integrations)

→ grpcio, grpcio-tools

→ **Utilities:**

→ grpcurl or Postman (with gRPC support)

→ Terminal / shell

→ Text editor or IDE (VS Code, PyCharm, etc.)

Setup Requirements

- Python 3.10+ and pip installed.
- Virtual environment created and activated (recommended).
- Required packages installed: langchain, grpcio, grpcio-tools.
- Starter file techdocs_monolith.py available in the working directory.
- (Optional) Access to OpenAI API key configured via environment variable.

Scenario

You're a developer at **TechDocs**, where a LangChain-based document Q&A system has grown from a small prototype with 50 users to a production candidate serving 5,000 daily users. The system currently runs as a **single Python script** with three tightly coupled components:

- A **document retriever** that queries a vector database
- An **LLM service** that calls OpenAI's API
- A **response summarizer** that post-processes outputs

Under heavy load:

- When the LLM times out, the **entire pipeline freezes**.
- Any code change requires **redeploying the whole script**, causing downtime.
- You **can't scale** the CPU-heavy retriever separately from the I/O-heavy LLM.
- A single component failure **crashes the entire app**.

Your team lead has decided to split the system into microservices that communicate via **gRPC**, and you've been assigned to prove the approach.

Objective

Design and implement the **first LangChain microservice** (retriever, LLM gateway, or summarizer) as a gRPC service, test it locally, and document why your chosen boundaries and interface make sense for future scaling and reliability.



Instructions for Learners

Getting Started

- 1 Open the starter file `techdocs_monolith.py` in your IDE.
- 2 Skim the code to understand how the retriever, LLM, and summarizer are currently wired together.
- 3 Confirm you can run the monolithic script end-to-end with a sample query.
- 4 Create a new folder (e.g., `retriever_service` or `summarizer_service`) for your microservice work.

Activity 1: Map the Monolith & Choose Your First Microservice

Objective

Identify clear service boundaries in the existing LangChain pipeline and choose one component to extract as a gRPC service.



Input/Data

→ techdocs_monolith.py

→ Sample query and demo documents configured in the starter script

Steps to Complete

Step 1: Trace the Data Flow

→ Locate the sections in the script that perform:

→ Document retrieval

→ LLM call

→ Summary / post-processing

→ Sketch (on paper or in notes) the sequence: query → retriever → LLM → summarizer → answer.

Step 2: Identify Responsibilities

→ For each component, jot down:

→ **Inputs:** What data it expects (e.g., user query, retrieved docs).

→ **Outputs:** What it returns (e.g., list of docs, raw LLM text, formatted summary).

→ Note which parts are CPU-bound (retriever) vs I/O-bound (LLM) vs logic-bound (summarizer).

Step 3: Choose One Component to Extract First

- Decide which component to turn into your **first microservice** (recommended: retriever or summarizer).
- Justify your choice in 2–3 bullet points (e.g., “retriever is heavily reused across endpoints”, “summarizer is easiest to isolate first”).

Step 4: Draft a gRPC Contract on Paper

- Sketch what your gRPC method might look like:
 - Request fields (e.g., query, top_k)
 - Response fields (e.g., documents, summaries)
- Identify data types (string, repeated string, etc.).

Expected Outcome:

A clear decision about **which component becomes your first microservice**, and an initial idea of its gRPC request/response shape.

Tips:

→ Choose a component that is logically self-contained and has a **clear input/output** boundary.

→ Don't overcomplicate the first interface—keep it simple and focused.

Activity 2: Design the .proto and Implement the gRPC Service

Objective

Create a .proto file for your chosen component, generate Python stubs, and implement a working LangChain-powered gRPC service.



Input/Data

- Notes from Activity 1 (chosen component + drafted interface)
- Starter LangChain logic from `techdocs_monolith.py`

Steps to Complete

Step 1: Create the .proto Definition

- Inside your service folder, create service.proto.
- Define:
 - A service block for your component (e.g., RetrieverService, SummarizerService).
 - A rpc method (e.g., GetDocuments, SummarizeAnswer).
 - message types for request and response with explicit fields.
- Keep messages typed and structured—avoid raw JSON blobs.

Step 2: Generate Python Stubs

→ Run `python -m grpc_tools.protoc` to generate Python code, for example:

→ `python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. service.proto`

→ Confirm that `_pb2.py` and `_pb2_grpc.py` files are created.

Step 3: Implement the gRPC Service Class

- Create server.py in your service folder.
- Import the generated stub classes and LangChain components.
- Implement the service class:
 - Map the rpc method to a LangChain pipeline that performs your chosen operation.
 - Convert from gRPC request → LangChain inputs → gRPC response.

Step 4: Start the gRPC Server

→ In server.py, create a gRPC server, add your service, and listen on a local port (e.g., localhost:50051).

→ Run the server and verify that it starts without errors.

Expected Outcome:

A running gRPC service that exposes one LangChain component as a standalone microservice, driven by a .protocontract.

Tips:

- Start with minimal fields in the .proto and expand later if needed.
- Keep the service stateless: don't store per-user state in memory between requests.

Activity 3: Test Your Service & Reflect on Boundaries

Objective

Validate the service from the outside using grpcurl or Postman and reflect on your design choices and trade-offs.



Input/Data:

- Running gRPC server from Activity 2
- Sample query or sample document from TechDocs

Steps to Complete

Step 1: Test with grpcurl or Postman

- Use a real request (e.g., user query) to call your gRPC method.
- Verify that:
 - The service responds without errors.
 - The response has the expected shape and fields.
 - The LangChain logic actually runs (retrieval/summarization, etc.).

Step 2: Capture Evidence

Save the grpcurl command and output, or a screenshot from Postman, showing a successful call.

Step 3: Write a Short Reflection (~200 words)

Reflect on:

Why you chose this component as the first microservice.

How your gRPC contract helps or hinders future scaling and evolution.

What trade-offs you see (e.g., extra network hop vs. independent scaling, potential schema rigidity).

Expected Outcome:

A **validated** microservice with at least one successful external test, and a concise written reflection on architectural decisions.

Tips:

→ Think of your reflection as something you'd share with your team lead to justify the approach.

→ Mention at least one benefit and one risk of your boundary choice.

Compile Your Findings

Requirement 1: gRPC Interface Contract

Question Prompt:

"Upload the .proto file for the LangChain component you extracted. What request and response schema did you define, and how does it reflect your service boundaries?"

Response Type:

File Upload

File Types:

Coding, Text, Audio, Video

→ .proto, .txt, .md, .json

→ .mp3, .wav, .aac

→ .mp4, .mov, .webm

Requirement 2: Microservice Implementation & Test Evidence

Question Prompt:

"Upload your Python microservice implementation and a screenshot or output from grpcurl/Postman showing a successful call. How does your test demonstrate that the service works correctly?"

Response Type:

File Upload

File Types:

Coding, Image, Text, Video

→ .py, .txt, .md

→ .png, .jpg, .jpeg, .webp

→ .mp4, .mov, .webm

Requirement 3: Boundary Reflection

Question Prompt:

"In 150–250 words, explain why you selected this specific component as your first microservice and how your gRPC boundary supports independent scaling and fault isolation."

Response Type:

Rich Text

File Types:

Not applicable (Rich Text submission)

Requirement 4: Limitations & Next Steps

Question Prompt:

"What limitations or risks does your current microservice design introduce, and what improvements or next steps would you recommend to strengthen the architecture?"

Response Type:

File Upload

File Types:

Text, Audio, Video

→ .txt, .md, .pdf

→ .mp3, .wav, .aac

→ .mp4, .mov, .webm

Summary

This activity provides hands-on experience with **splitting a monolithic LangChain application into microservices** using gRPC. You practiced:

- Reading and decomposing an existing LangChain script into logical components.
- Designing a **typed service contract** with .proto definitions.
- Implementing a **real gRPC microservice** around LangChain logic.
- Testing the service from the outside and reflecting on architectural trade-offs.

Key takeaways:

- Microservice boundaries should follow natural responsibility and data-flow lines, not arbitrary splits.
- A well-designed gRPC contract becomes the **source of truth** for how components talk to each other.
- Even one component extracted as a service can dramatically improve **scaling flexibility** and **fault isolation** in a LangChain system.

Resources

Primary Materials:

- techdocs_monolith.py starter script
- Your .proto file and server.py implementation

Reference Documentation:

- LangChain docs (Python)
- gRPC Python + grpcio-tools documentation

Additional Resources (Optional):

- grpcurl documentation or Postman gRPC guide
- Example LangChain + gRPC sample projects (if provided in the course repo)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.